Payment allocation: synthetic software breakdown

What can this tiny payment fixture establish, and what would a real system still need?

Summary

The original fixture keeps applied payment, remaining invoice balance and unapplied funds separate. This example demonstrates evidence-linked reporting, not a production payment integration.

Evidence labels describe the input. Validation checks structure and references; it does not verify the truth of a claim.

Findings

[OBSERVED] A partial payment of 3000 against a balance of 10000 leaves 7000 remaining and no unapplied funds. (fixture)

[OBSERVED] A payment of 15000 against a balance of 10000 applies 10000 and retains 5000 unapplied. (fixture)

[OBSERVED] The fixture rejects negative or fractional minor-unit values in the executed assertions. (fixture)

[OBSERVED] REA returned a structured static-analysis evidence envelope for the examples directory. (rea1)

Limit: This establishes a real inspection ran, not that REA proved every business rule or any production workflow.

Proposed requirements and acceptance

PAY-001: Preserve received, allocated and unapplied amounts separately.

Acceptance: Partial and overpayment cases retain the original amounts and reconcile the allocation totals.

PAY-002: Define duplicate-effect and reconciliation behavior before connecting a payment provider.

Acceptance: A provider timeout and repeated notification do not create duplicate money movement; durable provider readback resolves uncertainty.

Unknowns and remaining work

Provider authorization, capture, settlement, refund and dispute handling.

Persistent allocation records, concurrent updates, client isolation and audit history.

Jurisdiction-specific tax, accounting and payment obligations.

Human review is still required to assess whether findings are supported by their sources.

Evidence register

fixture — Original payment.mjs fixture and four executable assertions [available]; retrieved 2026-10-19T06:28:55.508238+00:00

artifact_sha256: 0fcf44a07bcc968f242805bd97362a0b80c002b8e8467f332c1a205efcec6fc1

scope: One original function; partial payment, overpayment, negative input and fractional input assertions ran through Node.

limitations: No processor, database, concurrent request, tax or invoice lifecycle is implemented.

rea1 — REA static JavaScript analysis of examples directory [partial]; retrieved 2026-10-19T06:28:55.511020+00:00

evidence_id: ev_27cb37699202e5e636f75bce85cd5fece7e60e6fc29c02603ca48a0789ed8833

scope: Inert syntax inspection through the actual REA CLI.

limitations: Static analysis does not execute the function or establish production behavior; full raw output is generated locally and not redistributed.